Chapitre 6 : Laravel

Siti Hajar
h.siti@emsi.ma

Plan

1. Installation et bases de Laravel
2. Structure d’un projet laravel
3. Réalisation d’une application CRUD avec laravel

Définition d’un framework

En génie logiciel et en programmation, un framework est un ensemble de composants logiciels réutilisables qui permettent de développer de nouvelles applications plus efficacement.

Un framework est un ensemble de bibliothèques, d'outils et de conventions qui impose une structure standardisée au développement logiciel. Il définit le squelette de l'application et gère les aspects techniques fondamentaux.

Un framework donne un cadre pour les projets informatiques.

Laravel

- Framework PHP open-source créé par Taylor Otwell en 2011
- Version actuelle : Laravel 12 (Février 2025)
- Framework PHP le plus populaire actuellement
- Basé sur le pattern MVC (Modèle-Vue-Contrôleur)

Les grands avantages de Laravel

1. **Simplicité de démarrage**
   a. Installation rapide avec une commande
   b. Structure déjà organisée
   c. Possibilité de coder immédiatement
2. **Fonctionnalités prêtes à l'emploi**
   a. Système de connexion/inscription intégré
   b. Gestion de base de données simple
   c. Système de templates pour les pages
   d. Protection contre les attaques web
3. **Outils pratiques**
   a. Artisan : créer du code automatiquement
4. **Base de données facilitée**
   a. Pas de SQL complexe à écrire
   b. Relations entre tables simplifiées
   c. Manipulation des données en peu de code

PHP natif vs Laravel

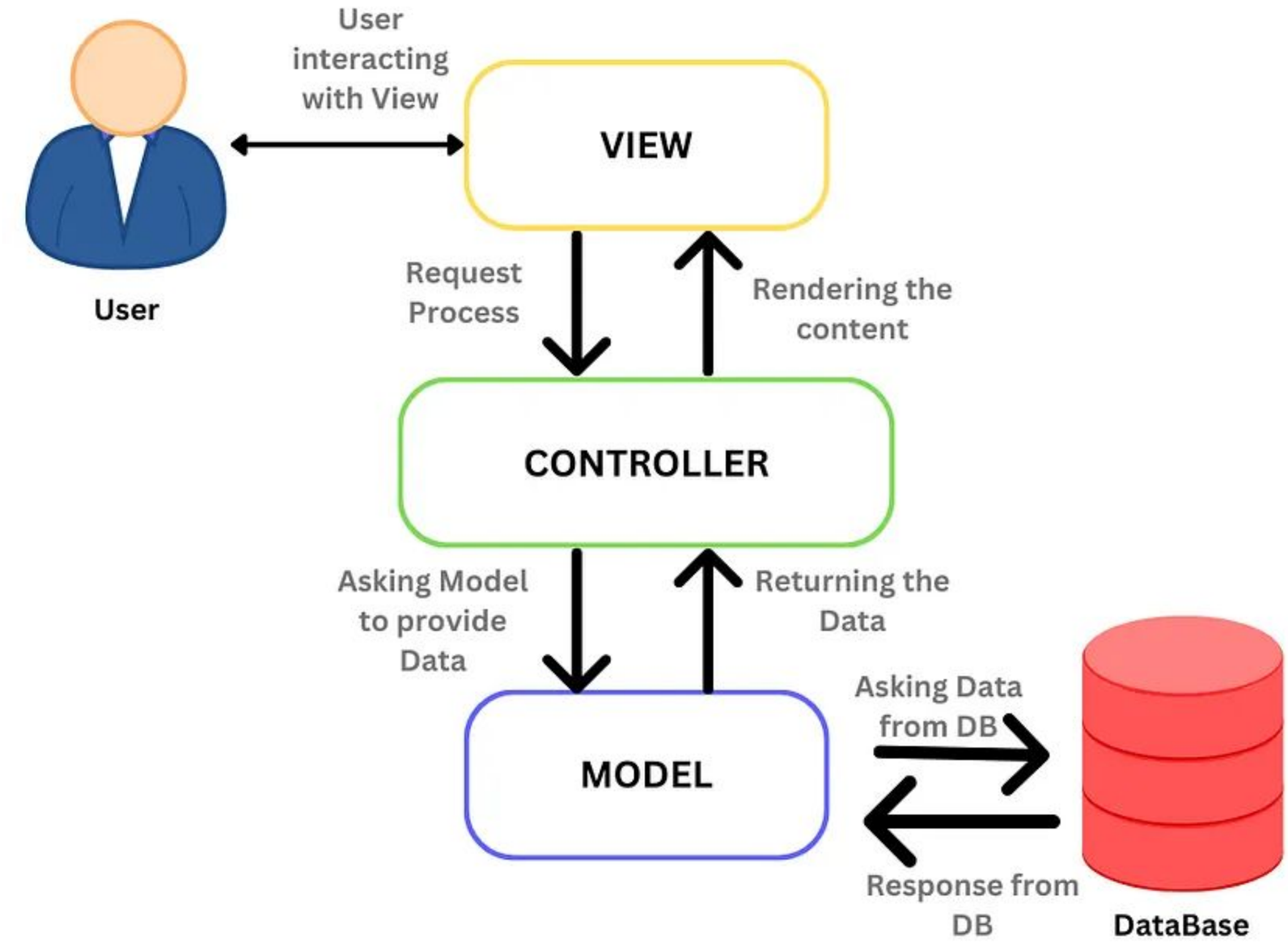
Sans Laravel (PHP pur)

- Vous devez écrire tout le code vous-même
- Vous devez gérer la sécurité vous-même
- Vous devez organiser vos fichiers vous-même

Avec Laravel

- Beaucoup de fonctionnalités sont déjà prêtes à l'emploi
- La sécurité est déjà en place
- Les fichiers sont bien organisés

L’architecture MVC



Environnement de Développement Laravel pour la version 12

PHP Version 8.2 minimum

- Gestionnaire de dépendances PHP
- Installation facile depuis getcomposer.org

Vs code

XAMPP

node js

Définition de Composer

Composer est un gestionnaire de dépendances pour PHP qui permet de définir des paquets de code tiers utilisés par un projet et qui peuvent ensuite être facilement installés et mis à jour.

Composer est un gestionnaire de dépendances pour PHP. C’est un outil simple que les développeurs utilisent pour gérer et intégrer des paquets ou des bibliothèques externes dans leurs projets basés sur PHP.

Création d’un projet laravel avec composer

composer create-project laravel/laravel app-name

télécharge et installe les bibliothèques nécessaires au projet

C'est la commande de Composer qui crée un nouveau projet

le nom du package Laravel qui contient les fichiers et structures de base nécessaires pour démarrer une application Laravel.

Création d’un projet laravel avec laravel

```
laravel new app-name
```

La structure d’un projet laravel

1. **Dossiers Principaux du Frontend**
   a. **/resources/views/ :** Contient toutes les vues Blade
   b. **/resources/css/ :** Contient les fichiers CSS
   c. **/resources/js/** - Contient les fichiers js
2. **Dossiers Backend Laravel**
   a. **/app**
      i. **/Http/Controllers** - Contrôleurs qui renvoient des réponses
      ii. **/Models** - Modèles Éloquent pour la base de données
   b. **/routes**
      i. **web.php** - Définit les routes qui renvoient des pages web
3. **Configuration**
   a. **vite.config.js** - Configuration de Vite
   b. **.env** - Variables d'environnement
   c. **package.json** - Dépendances JavaScript/React
   d. **composer.json** - Dépendances PHP/Laravel

Composer

Composer est un gestionnaire de dépendances pour PHP, créé en 2012. Il permet de gérer facilement les bibliothèques et les packages dont un projet PHP a besoin.

Le fonctionnement de Composer est basé sur un fichier composer.json qui liste toutes les dépendances du projet.

- **Les principales commandes de composer:**

```
composer create-project laravel/laravel nom-projet      # Créer un nouveau projet Laravel
composer require package                                # Installer une dépendance
composer install                # Installer les dépendances du projet (depuis composer.json)
composer update                 # Mettre à jour toutes les dépendances
composer update package         # Mettre à jour une dépendance spécifique
composer show                   # Voir les packages installés
composer outdated               # Vérifier les dépendances qui peuvent être mises à jour
```

PHP Artisan

PHP Artisan est l'interface en ligne de commande (CLI) incluse avec Laravel. Au lieu de créer manuellement des fichiers et d'écrire du code répétitif, Artisan peut le faire via des commandes simples.

- **Démarrer le serveur local:**

  ```
  php artisan serve
  ```

- **Création de fichiers:**

  ```
  php artisan make:controller UserController        # Créer un contrôleur
  php artisan make:model Product -m                 # Créer un modèle et sa migration
  php artisan make:migration create_users_table     # Créer une migration
  ```

- **Gestion de la base de données:**

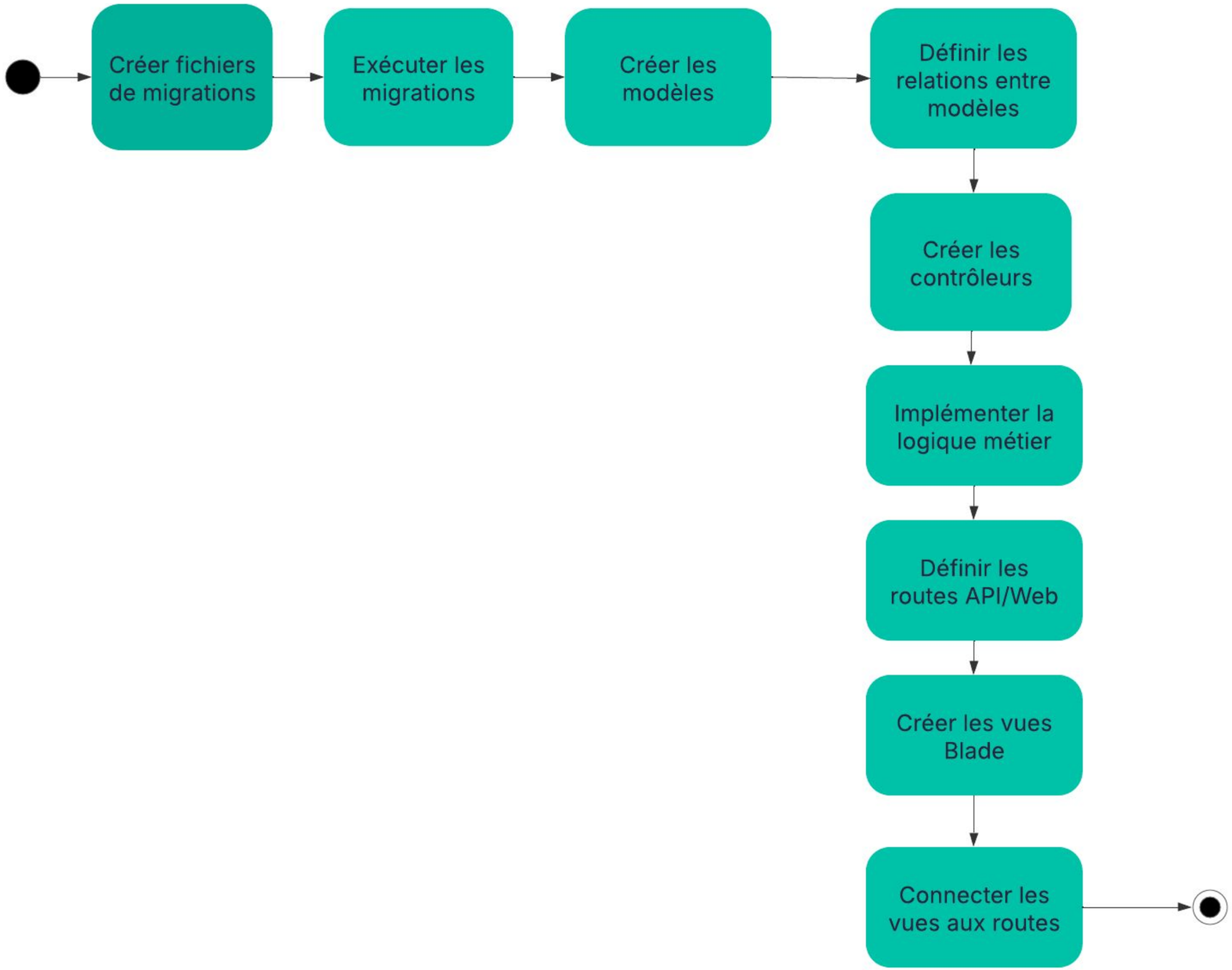
  ```
  php artisan migrate                               # Exécuter les migrations
  php artisan migrate:rollback                      # Annuler la dernière migration
  php artisan migrate:fresh                         # Rafraîchir la base de données
  php artisan db:seed                               # Exécuter tous les seeders
  ```

Lancer le projet

La commande php artisan serve :

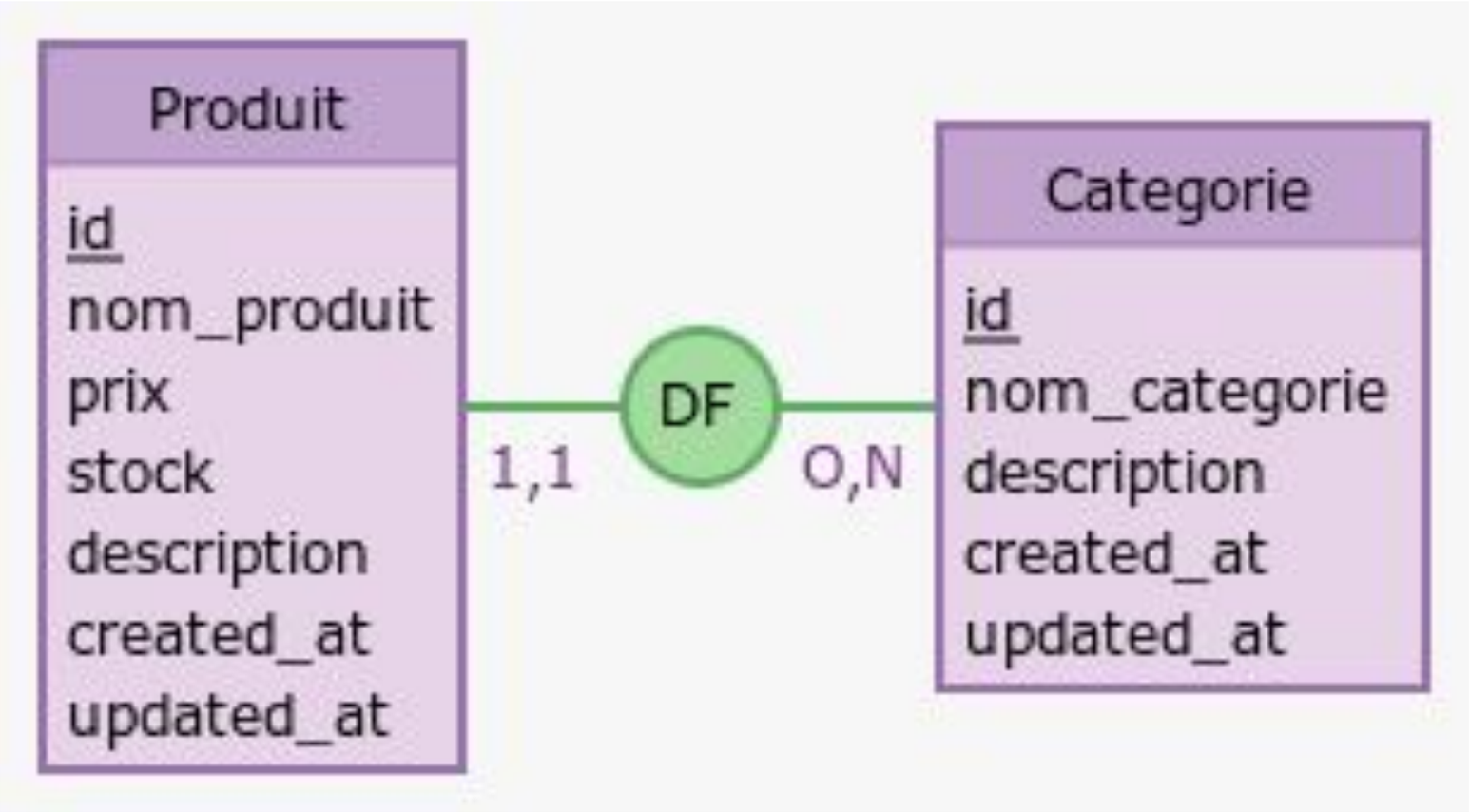
- Lance un serveur de développement local
- Rend le projet accessible à l'adresse http://localhost:8000

Ce qu'on va réaliser



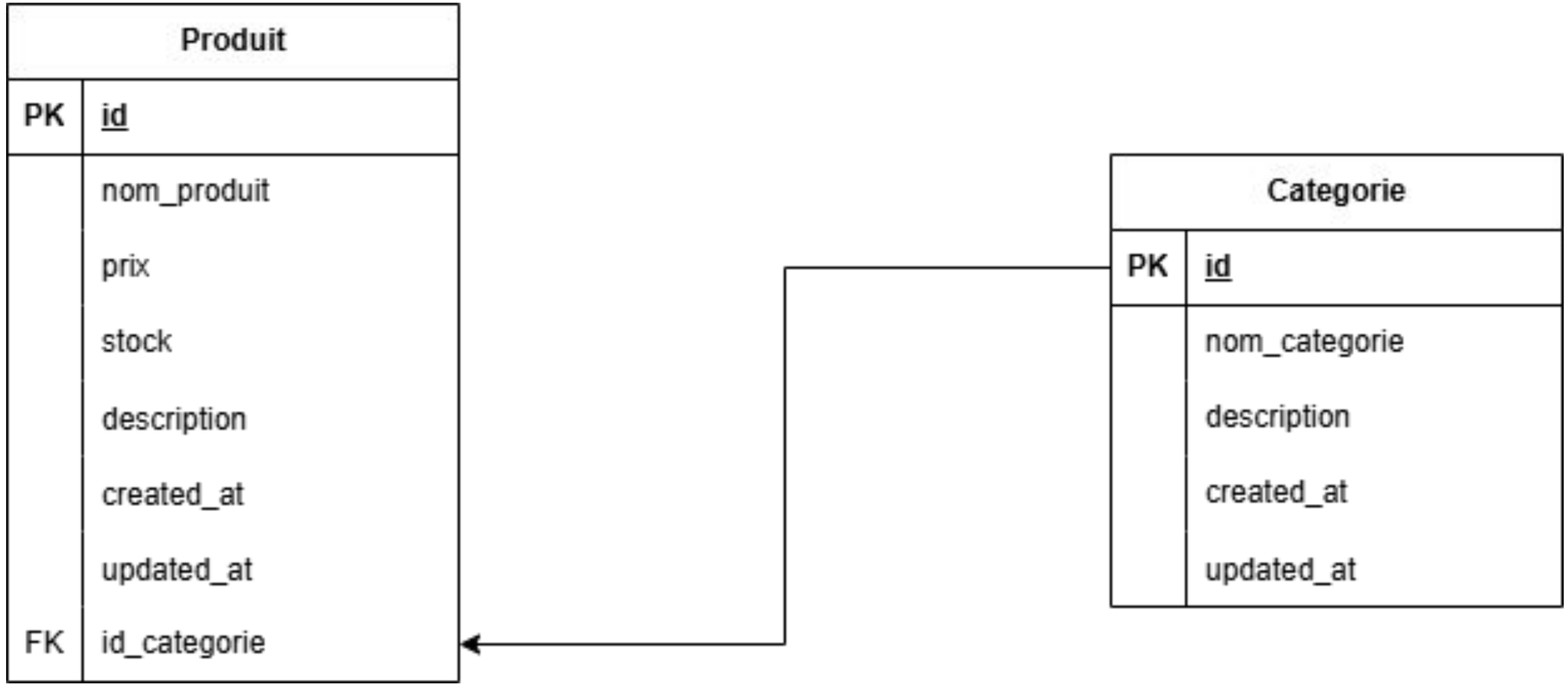
Etape 1: Database

MCD



Etape 1: Database

MLD



Etape 1: Database

1. Création de la Migration:

Une migration est un fichier PHP qui définit les changements à apporter à la structure de la base de données. Elle peut créer, modifier ou supprimer des tables et des colonnes.

Les migrations dans Laravel sont ajoutées dans le dossier **database/migrations** du projet. Ce dossier contient tous les fichiers de migration, chacun correspondant à une modification spécifique de la structure de la base de données.

Fonctionnalités des Migrations :

- Définir la structure des tables
- Modifier des tables existantes
- Créer des clés primaires et étrangères
- Définir des index
- Gérer les types de données

Etape 1: Database

1. Création de la Migration:

php artisan make:migration create_categories_table

La commande artisan qui génère un nouveau fichier de migration

C'est le nom de la migration.
Laravel va donc générer un fichier qui contient deux fonctions **up()** et **down()**, et comprend qu'on est en train de créer une migration pour une table dans la base de données. Le nom de cette table correspond au mot situé entre **create** et **table** dans le nom du fichier. Dans notre cas, c'est la table **categories**.

Etape 1: Database

1. Création de la Migration:

Le fichier généré contient par défaut le code suivant:

```php
<?php

use Illuminate\Database\Migrations\ Migration;
use Illuminate\Database\Schema\ Blueprint;
use Illuminate\Support\Facades\ Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('categories');
    }
};
```

Ce fichier contient une classe anonyme qui hérite de la classe Migration prédéfinie dans Laravel.
Cette classe importe trois éléments essentiels :

- **Migration** qui fournit la structure de base;
- **Blueprint** qui permet de définir la structure des tables;
- **Schema** qui est la façade permettant d'interagir avec la base de données.

La classe contient deux méthodes principales.

- **La méthode up()** est exécutée lors de l'application de la migration et utilise Schema::create() pour créer la table **categories** dans la base de données. À l'intérieur de cette méthode, on définit la structure de la table
- **La méthode down()** fait l'inverse : elle est exécutée lors d'un rollback (annulation de la migration) et utilise Schema::dropIfExists() pour supprimer la table **categories** de la base de données si elle existe.

Etape 1: Database

Les méthodes Blueprint les plus courantes avec leur équivalent MySQL : (Pour des informations détaillées consultez le site: https://laravel.com/docs/12.x/migrations)

Méthode	Description	Équivalent MySQL
id()	Clé primaire auto-incrémentée	BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
string('nom')	Chaîne de caractères (255 par défaut)	VARCHAR(255)
string('nom', 100)	Chaîne avec longueur personnalisée	VARCHAR(100)
text('description')	Texte long	TEXT
integer('age')	Nombre entier	INT
boolean('actif')	Booléen (vrai/faux)	TINYINT(1)
decimal('prix', 8, 2)	Nombre décimal (8 chiffres, 2 après virgule)	DECIMAL(8, 2)
timestamps()	Crée created_at et updated_at	TIMESTAMP (x2)
foreignId('user_id')	Clé étrangère	BIGINT UNSIGNED
foreignId('user_id')->constrained()	Clé étrangère avec contrainte automatique	BIGINT UNSIGNED + FOREIGN KEY (user_id) REFERENCES users(id)

Etape 1: Database

Définir la structure de la table categories:

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();
            $table->string('category_name');
            $table->string('description');
            $table->timestamps();
        });
    }
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('categories');
    }
};
```

Etape 1: Database

Création de la 2eme migration pour la table Produits: php artisan make:migration create_products_table

```php
<?php
use Illuminate\Database\Migrations\ Migration;
use Illuminate\Database\Schema\ Blueprint;
use Illuminate\Support\Facades\ Schema;
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('product_name');
            $table->string('description');
            $table->decimal('price', 6, 2);
            $table->foreignId('caterory_id')
                  -> constrained()
                  -> restrictOnDelete()
                  -> cascadeOnUpdate();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

Etape 1: Database

Commandes pour gérer les migrations:

- **Créer une migration:**

  php artisan make:migration create_tableName_table #exécute la méthode up

- **Exécuter les migrations:**

  php artisan migrate

- **Annuler une migration:**

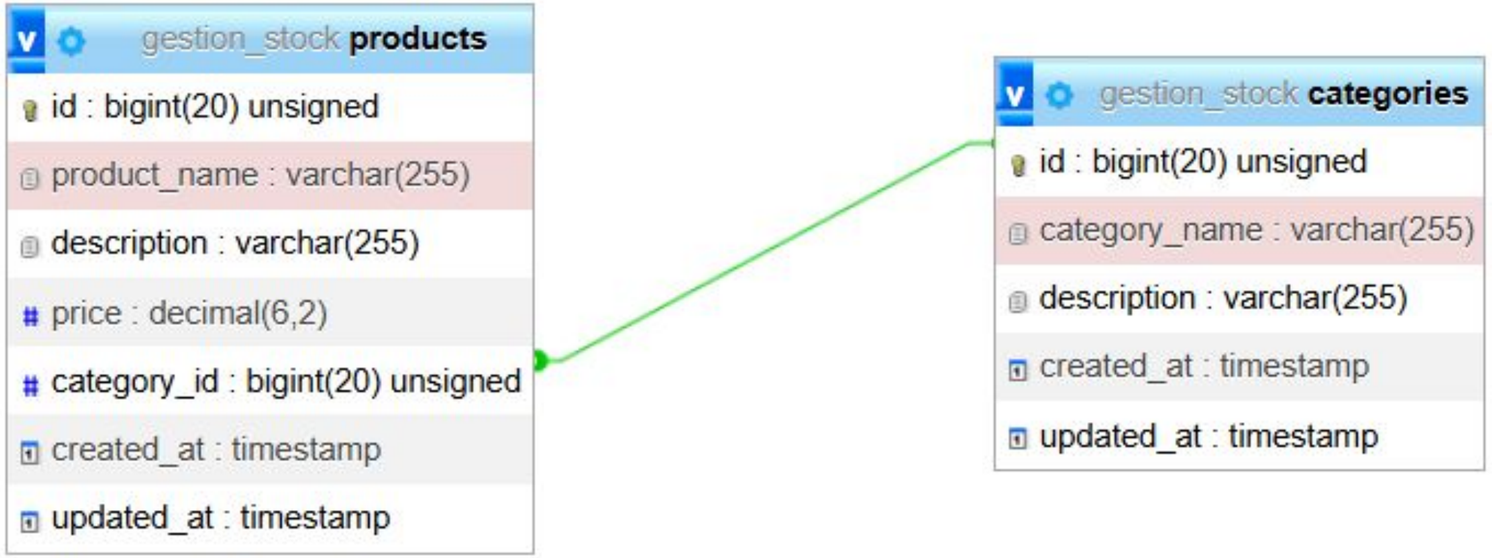
  php artisan migrate:rollback #exécute la méthode down

- **Ré-exécuter les migrations:**

  php artisan migrate:refresh

Etape 1: Database

Après l'exécution des migrations les deux tables doivent être crée dans la base de données:



Etape 1: Database

Mise à jour d'une table existante dans la base de données:

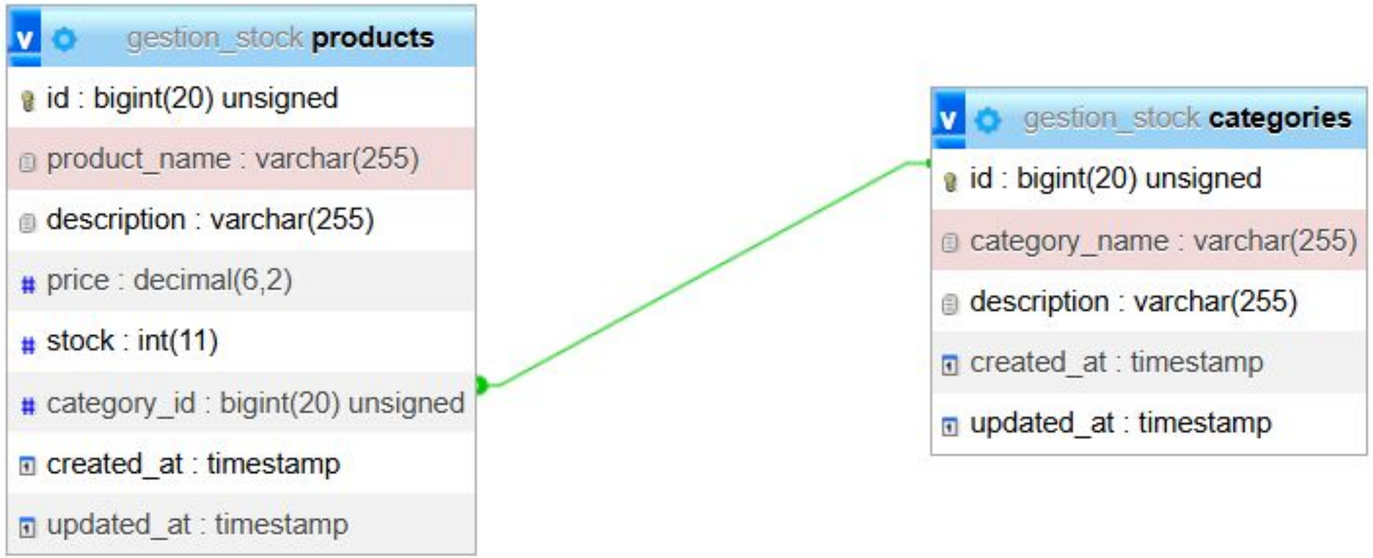
Créer une migration: php artisan make:migration add_columns_to_products_table

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('products', function (Blueprint $table) {
            $table->integer('stock')->after('price');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('products', function (Blueprint $table) {
           $table->dropColumn('stock');
        });
    }
};
```

- **Schema::table()** est une méthode Laravel utilisée pour modifier une table existante dans la base de données, contrairement à Schema::create() qui sert à créer une nouvelle table.
- **La méthode up()** contient les modifications à appliquer lors de l'exécution de la migration.
- **La méthode down()** définit comment annuler les modifications si nous faisons un rollback de cette migration.

Etape 1: Database

Après l'exécution de la dernière migration (en utilisant la commande php artisan migrate) la colonne stock est ajoutée à la table products juste après la colonne price.



Etape 1: Database

2. Création du Model:

Un modèle est une classe PHP qui représente une table de la base de données. Il agit comme un pont entre la base de données et le code, facilitant la manipulation des données (lecture, écriture, mise à jour, suppression).

Les modèles dans Laravel se trouvent dans le dossier **app/Models.**

- Chaque modèle est associé à une table spécifique dans la base de données.
- Un modèle permet d'interagir avec les données en utilisant des méthodes simples, évitant d'écrire des requêtes SQL complexes.
- Laravel s'appuie sur un système appelé Eloquent ORM (Object-Relational Mapping), qui permet de travailler directement avec des objets au lieu de manipuler des tables de base de données.

Etape 1: Database

2. Création du Model:

Eloquent ORM(Object-Relational-Mapping): Eloquent est l'implémentation par Laravel du pattern **Active Record**. Chaque table de base de données possède un Modèle correspondant qui est utilisé pour interagir avec cette table. En plus de récupérer des enregistrements de la table, les modèles Eloquent vous permettent également d'insérer, de mettre à jour et de supprimer des enregistrements.

Active Record Pattern:

Eloquent implémente le pattern Active Record, où chaque instance de modèle représente une ligne de la table et contient à la fois les données et la logique pour interagir avec la base de données.

Etape 1: Database

2. Création du Model Category:

php artisan make:model Category

La commande artisan qui génère un nouveau fichier du modèle Eloquent.

Le nom du modèle à créer.
Par convention dans Laravel, lors de la création d'un modèle nommé **Category** (au singulier), Laravel suppose automatiquement que ce modèle correspond à une table nommée **categories** (au pluriel, en minuscules) dans la base de données.

Etape 1: Database

Définir la structure du modèle category:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    protected $fillable = [
        'category_name',
        'description'
    ];
    public function products(){
        return $this->hasMany(Product::class);
    }
}
```

- **$fillable** est une propriété protégée qui définit une liste blanche des colonnes de la table qui peuvent être remplies en masse.
- **products()** définit la relation : une catégorie a plusieurs produits via Eloquent.

Etape 1: Database

2. Création du Model Product:

php artisan make:model Product

La commande artisan qui génère un nouveau fichier du modèle Eloquent.

Le nom du modèle à créer.
Par convention dans Laravel, lors de la création d'un modèle nommé **Product** (au singulier), Laravel suppose automatiquement que ce modèle correspond à une table nommée **products** (au pluriel, en minuscules) dans la base de données.

Etape 1: Database

Définir la structure du modèle category:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    protected $fillable = [
        'product_name',
        'price',
        'stock',
        'description',
        'category_id'
        ];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }
}
```

- **$fillable** est une propriété protégée qui définit une liste blanche des colonnes de la table qui peuvent être remplies en masse.
- **category()** définit la relation : un produit appartient à une catégorie via Eloquent.

Etape 1: Database

les principaux attributs disponibles dans les modèles Laravel:

- **$fillable**: Liste les champs autorisés pour l'assignation de masse. Tous les champs listés peuvent être remplis via create() ou update() **=> protected $fillable = ['nom', 'email', 'password'];**
- **$guarded:** Inverse de $fillable. Liste les champs protégés contre l'assignation de masse. Si vide [ ], tous les champs sont assignables. **=> protected $guarded = ['id', 'admin'];**
- **$table**: Spécifie le nom de la table si différent de la convention Laravel (singulier du nom de la table).

  =>protected $table = 'mes_utilisateurs';

- **$timestamps**: Active/désactive les colonnes created_at et updated_at **=> public $timestamps = false;**
- **$attributes**: Valeurs par défaut pour les attributs. **=> protected $attributes = ['statut' => 'actif', 'role' => 'user'];**

Etape 1: Database

Les méthodes Eloquent fréquemment Utilisées:

Methode	Utilisation	Description
all()	User::all()	Récupère tous les enregistrements de la table
find($id)	User::find(3)	Trouve un enregistrement par sa clé primaire
get()	User::where('age', '>', 20)->get() User::orderBy('created_at', 'desc')->get()	Exécute la requête et récupère les résultats
create($array)	User::create(['name' => 'Hajar’'])	Crée et sauvegarde un nouvel enregistrement
save()	$user->save()	Sauvegarde le modèle en base de données
update($array)	User::where('active', 0)->update(['active' => 1])	Met à jour les enregistrements correspondants
delete()	$user->delete()	Supprime l'enregistrement
destroy($ids)	User::destroy(1) ou User::destroy([1,2,3])	Supprime par clé(s) primaire(s)
count()	User::count()	Compte les enregistrements
exists()	User::where('email', $email)->exists()	Vérifie si des enregistrements existent

Etape 1: Database

Les méthodes de relations fréquemment Utilisées:

Methode	Utilisation	Description
hasOne()	return $this->hasOne(Passport::class)	Définit une relation un-à-un (One-to-One)
hasMany()	return $this->hasMany(Post::class)	Définit une relation un-à-plusieurs (One-to-Many)
belongsTo()	return $this->belongsTo(User::class)	Définit une relation inverse (plusieurs-à-un)
belongsToMany()	return $this->belongsToMany(Role::class)	Définit une relation plusieurs-à-plusieurs
with($relations)	User::with('posts')->get()	Charge les relations

Etape 2: Backend

1. Création du Controller :

php artisan make:controller ProductController --resource

Un **controller** est une classe qui gère la logique entre les **requêtes utilisateur** et les **réponses** de l'application. Il sert de lien entre les routes, les modèles, et les vues.

Controllers dans Laravel sont ajoutées dans le dossier **app/Http/Controllers**

Commande pour créer un contrôleur et l'associer à un modèle avec les opérations CRUD :

php artisan make:controller ProductController --resource --model=Product

- --resource : génère automatiquement les méthodes CRUD (index, create, store, show, edit, update, destroy)
- --model=NomModel: associe le contrôleur au modèle spécifié

Etape 2: Backend

Définir la structure du controller: php artisan make:controller CategoryController --resource --model=Category

```php
<?php

namespace App\Http\Controllers;

use App\Models\Category;
use Illuminate\Http\Request;

class CategoryController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $categories = Category::all();
        return view("categories.index", compact('categories'));
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        return view("categories.create");
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        Category::create($request->all());
        return redirect()->route('categories.index');
    }
```

La suite

```php
/**
     * Display the specified resource.
     */
    public function show(Category $category)
    {
       return view('categories.show', compact('category'));
    }

    /**
     * Show the form for editing the specified resource.
     */
    public function edit(Category $category)
    {
        return view("categories.edit", compact('category'));
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, Category $category)
    {
        $category->update($request->all());
        return redirect()->route("categories.index");

    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(Category $category)
    {
        $category->delete();
        return redirect()->route("categories.index");
    }
}
```

Etape 2: Backend

Définir la structure du controller: php artisan make:controller ProductController --resource --model=Product

```php
<?php

namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $products = Product::all();
        return view("products.index", compact('products'));
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        return view("products.create");
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        Product::create($request->all());
        return redirect()->route("products.index");
    }
```

La suite

```php
    /**
     * Display the specified resource.
     */
    public function show(Product $product)
    {
        return view("products.show", compact('product'));
    }

    /**
     * Show the form for editing the specified resource.
     */
    public function edit(Product $product)
    {
        return view("products.edit", compact('product'));
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, Product $product)
    {
        $product->update($request->all());
        return redirect()->route("products.index");
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(Product $product)
    {
        $product->delete();
        return redirect()->route("products.index");
    }
}
```

Etape 2: Backend

Les 7 méthodes du controller Category:

Méthode	Description	Route vers laquelle elle mène
index()	Affiche la liste de toutes les catégories	categories/index
create()	Affiche le formulaire de création d'une nouvelle catégorie	categories/create
store()	Enregistre une nouvelle catégorie dans la base de données	rediriger vers: categories/index
show()	Affiche les détails d'une catégorie spécifique	categories/show
edit()	Affiche le formulaire de modification d'une catégorie existante	categories/edit
update()	Met à jour une catégorie existante dans la base de données	rediriger vers: categories/index
destroy()	Supprime une catégorie de la base de données	rediriger vers: categories/index

Etape 2: Backend

La validation des données envoyées par l'utilisateur à partir des formulaires avant de les sauvegarder en base de données (Dans les méthodes update et store)

La validation protège l’application contre plusieurs risques. Elle empêche l'injection de code malveillant, évite les erreurs de type de données qui pourraient causer des bugs.

la méthode validate(): simplifie considérablement le processus de validation. Cette méthode s'applique directement sur l'objet Request et accepte un tableau de règles de validation. Elle vérifie automatiquement les données, gère les erreurs, et redirige l'utilisateur vers la page précédente en cas d'échec. Laravel examine chaque champ selon les règles définies. Si toutes les validations réussissent, la méthode retourne un tableau contenant uniquement les données validées.

Etape 2: Backend

Exemple de validation dans la fonction store de ProductController:

```
public function store(Request $request){
        $validated = $request->validate([
            'product_name' => 'required|string|max:255',
            'description' => 'required|string|max:1000',
            'price' => 'required|numeric|min:0|max:9999.99',
            'category_id' => 'required|exists:categories,id',
        ]);

        Product::create($validated);

        return redirect()
            ->route("products.index")
            ->with('success', 'Produit créé avec succès!');
    }
```

Etape 2: Backend

Tableau des règles de validation fréquemment utilisées: pour plus d'informations visitez le lien suivant:

https://laravel.com/docs/12.x/validation#available-validation-rules

Règle	Description	Exemple d’utilisation
required	Champ obligatoire	'name' => 'required'
nullable	Peut être null	'bio' => 'nullable|string'
string	Doit être une chaîne	'name' => 'string'
numeric	Doit être un nombre (float ou int)	'age' => 'numeric'
email	Doit être un email valide	‘email’ => ‘email’
min:X	Valeur/longueur minimum	'password' => 'min:8’
max:X	Valeur/longueur maximum	'name' => 'max:255'
size:X	Taille exacte	'code' => 'size:6'
exists:table,column	Existe en base de données	'category_id' => 'exists:categories,id'
regex:pattern	Expression régulière	'phone' => 'regex:/^[0-9]{10}$/'
in:liste	Dans une liste	'role' => 'in:admin,user,guest'

Etape 2: Backend

2. Configuration des Routes:

Les **routes** définissent les URL de l’application et indiquent ce qui doit se passer quand un utilisateur accède à une de ces URL.

Les routes sont définies dans le dossier routes, qui contient plusieurs fichiers spécifiques pour différents types de fonctionnalités.

- **web.php:** contient les routes destinées aux pages web accessibles via un navigateur.
- **api.php:** contient les routes pour les API REST. Les routes définies ici sont utilisées par des applications front-end (React, Vue...) ou d'autres systèmes.
- **console.php:** contient les définitions des commandes Artisan personnalisées.
- **channels.php:** gère les canaux pour les notifications en temps réel via Laravel Echo et Pusher.

Etape 2: Backend

2. Configuration des Routes:

Ajoutons les routes dans **routes/web.php:**

```
Route::get('/products', [ProductController::class, 'index'])->name('products.index');
Route::get('/products/create', [ProductController::class, 'create'])->name('products.create');
Route::post('/products', [ProductController::class, 'store'])->name('products.store');
Route::get('/products/{product}', [ProductController::class, 'show'])->name('products.show');
Route::get('/products/{product}/edit', [ProductController::class, 'edit'])->name('products.edit');
Route::put('/products/{product}', [ProductController::class, 'update'])->name('products.update');
Route::delete('/products/{product}', [ProductController::class, 'destroy'])->name('products.destroy');
```

Etape 2: Backend

2. Configuration des Routes:

Route::resource() est un raccourci qui génère automatiquement les 7 routes RESTful pour un contrôleur.

```
Route::resource('categories', CategoryController::class);
Route::resource('products', ProductController::class);
```

Etape 2: Backend

Les 7 routes générées automatiquement:

Verbe HTTP	URI	Nom de la route	Description
GET	/products	products.index	Récupérer/Afficher des données
GET	/products/create	products.create	Récupérer/Afficher des données
POST	/products	products.store	Envoyer/Créer de nouvelles données
GET	/products/{product}	products.show	Récupérer/Afficher des données
GET	/products/{product}/edit	products.edit	Récupérer/Afficher des données
PUT/PATCH	/products/{product}	products.update	Modifier des données existantes
DELETE	/products/{product}	products.destroy	Supprimer des données